13 ноября 2026

Лабораторная работа 2

# Skiplist

Сычев Ефим Андреевич, Б02-506

3 семестр

# 1 Реализация списка с пропусками

В ходе выполнения работы был написан шаблонный класс `SkipList<T, Compare>`, реализующий список с пропусками с возможностью задания своего компаратора (по умолчанию используется `std::less<T>`). Каждый элемент списка представлен структурой `Node`, в которой хранится само значение `value`, указатель `prev` на предыдущий узел нижнего уровня и вектор `std::vector<Node*> next` с указателями на следующие элементы на каждом из уровней. В начале и в конце списка создаются фиктивные узлы `head` и `tail`, за счет которых не нужно отдельно проверять выход за границы при обходе и можно сдвигать итератор `end()` назад.

Случайная высота нового узла при вставке выбирается с помощью генератора `std::mt19937` и равномерного распределения `std::uniform_real_distribution<double>(0.0, 1.0)`. Новый элемент всегда добавляется на нижний уровень и с вероятностью $p$ поднимается на уровень выше, пока случайная величина не выпадет больше $p$ или высота не достигнет $K + 1$, где $K$ — текущее число уровней `max_level`.

В классе реализованы конструктор по умолчанию, конструктор с передачей компаратора и вероятности $p$, а также шаблонный конструктор от пары итераторов `[first, last)`, который позволяет заполнить список из другого контейнера (например, `std::unordered_set`). Копирующий конструктор и оператор присваивания поэлементно копируют значения во вновь созданный список, а перемещающий конструктор и перемещающее присваивание работают за $O(1)$, обменивая указатели `head`, `tail` и поля класса через метод `swap()`. Количество элементов хранится в переменной `elem_count`, поэтому методы `size()` и `empty()` также выполняются за $O(1)$.

Для обхода элементов в отсортированном порядке написан вложенный класс `iterator` с категорией `std::bidirectional_iterator_tag`, который двигается по нижнему уровню через ссылки `next[0]` и `prev`. Обратный итератор `reverse_iterator` получен через стандартный адаптер `std::reverse_iterator<iterator>`.

Поиск позиции ключа выполняется методами `lower_bound` и `upper_bound`, которые спускаются с верхнего уровня вниз за $O(\log N)$ и возвращают итератор на первый элемент, не меньший или строго больший заданного ключа. Через них выражены методы `find`, `equal_range` и `count`. При вставке (`insert`) одинаковые элементы ставятся после уже добавленных, поэтому `find` всегда находит первый из дубликатов. Метод удаления `erase` (по одному итератору и по диапазону) ищет удаляемый узел по совпадению указателя `Node*` и перекидывает ссылки соседних узлов мимо него, благодаря чему остальные дубликаты и старые итераторы на другие элементы не ломаются.

# 2 Тесты и замеры времени

Для проверки времени работы вставки, поиска и удаления использовался таймер `std::chrono::steady_clock` (программа компилировалась с флагом оптимизации `-O2`). Перед каждым запуском массив из $N$ чисел от 0 до $N-1$ перемешивался в случайном порядке,

после чего по очереди замерялось среднее время одной операции `insert`, `find` и `erase` (удаление проводилось по заранее сохраненным итераторам, чтобы не учитывать время повторного поиска). Графики полученных зависимостей показаны на рис. 1 и 2:

- **Зависимость от числа элементов** $N$ **(**$p = 0.5$**):** при увеличении размера списка в точках среднее время операций растет: для вставки от $\approx 250$ нс ($N = 1000$) до $\approx 1200$ нс ($N = 500000$), для поиска от $\approx 100$ нс до $\approx 1200$ нс, а для удаления от $\approx 150$ нс до $\approx 1000$ нс.

- **Зависимость от вероятности** $p$ **(**$N = 100\,000$**):** наблюдается U-образная зависимость. При малых значениях ($p = 0.05$) время поиска достигает $\approx 2200$ нс (вставка $\approx 1450$ нс, удаление $\approx 1800$ нс). В диапазоне оптимума $p \in [0.3, 0.7]$ время стабилизируется и составляет $\approx$ 400–600 нс для всех операций. При росте до $p = 0.95$ время вставки увеличивается до $\approx 1100$ нс (поиск $\approx 1000$ нс, удаление $\approx 900$ нс) из-за избыточной высоты узлов.

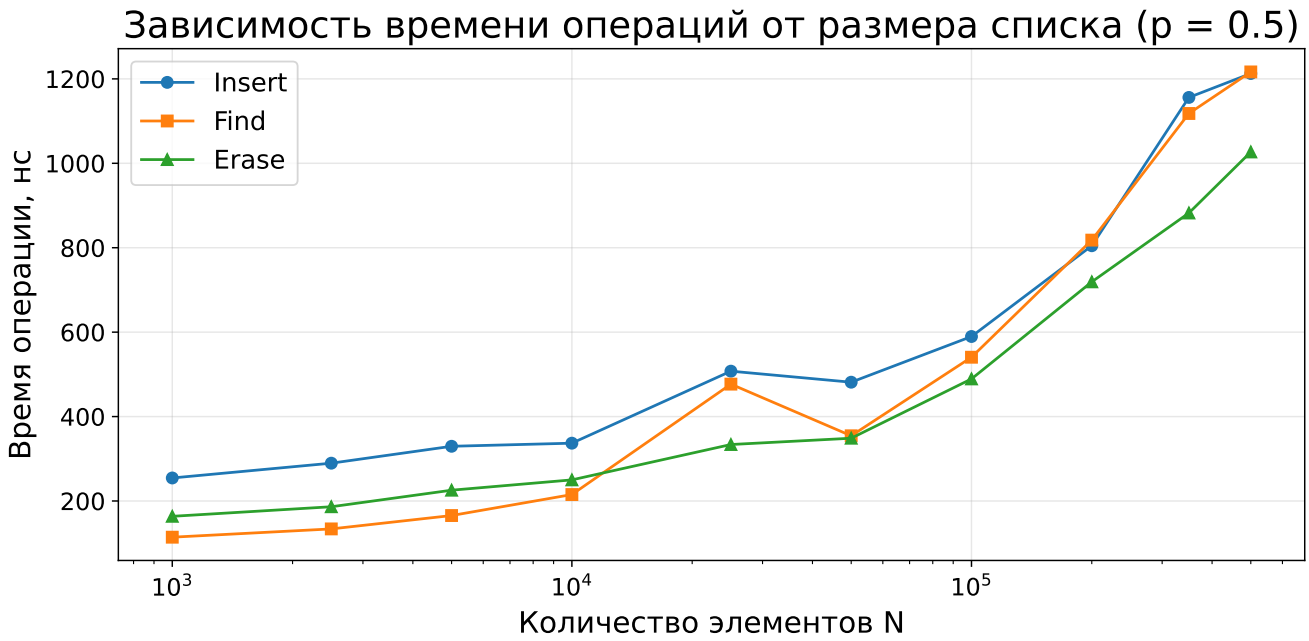



Рис. 1: Зависимость среднего времени операций `insert`, `find` и `erase` от количества элементов $N$ при $p = 0.5$

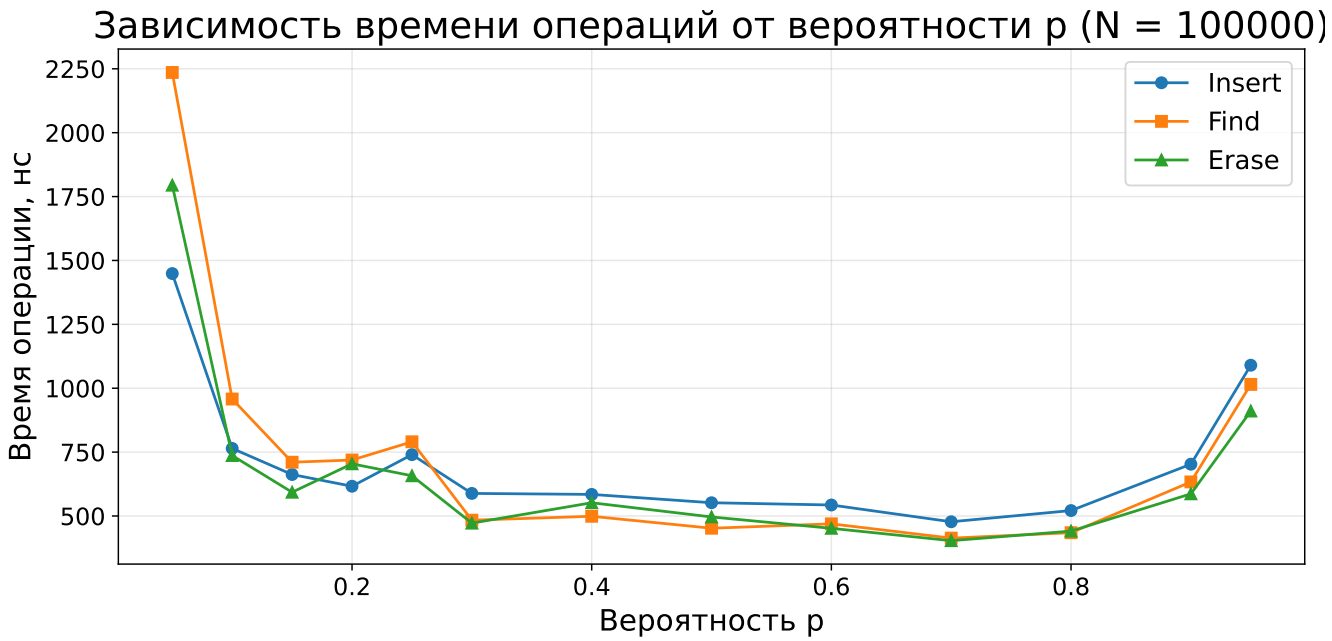



Рис. 2: Зависимость среднего времени операций `insert`, `find` и `erase` от вероятности $p$ при $N = 100\,000$